

A NUMBER I ACTUALLY MEASURED

~20%

faster p95 from one cache.

The pattern — and the one failure mode I still haven't tested.

CONTEXT

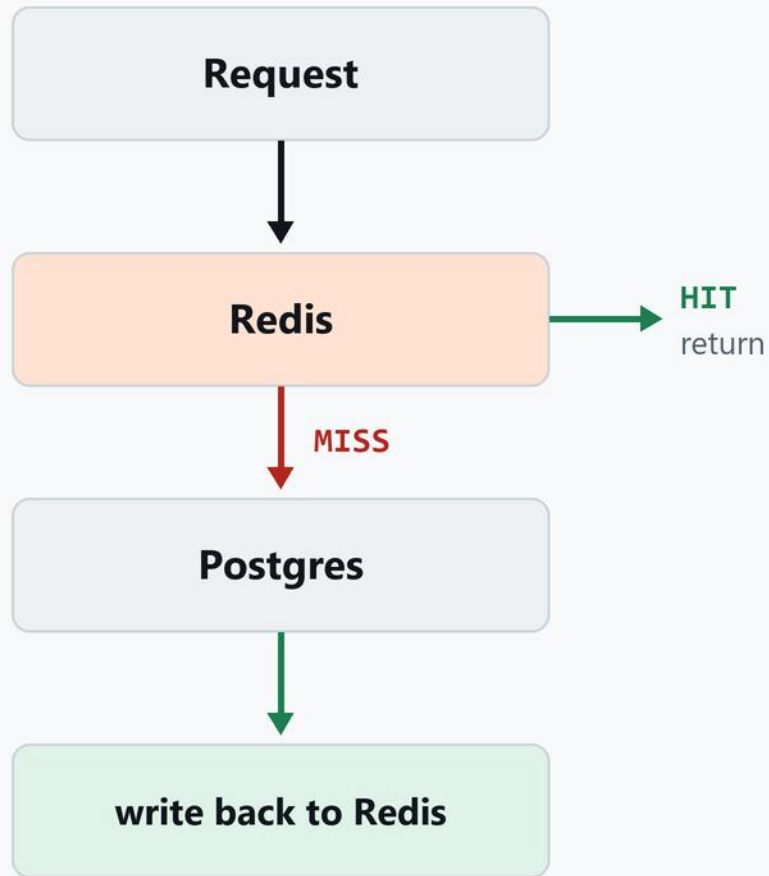
A system moving 10M+ messages a month.

Every millisecond on the read path multiplies.

WHAT p95 MEANS

The response time that 95% of requests come in under.

Not the average. Averages hide the worst realistic case — p95 is closer to what your slowest real users actually feel.



Hit? Skip the database entirely.
Miss? Pay once, then cache it.

p95 on that endpoint

BEFORE

baseline

AFTER • REDIS CACHE-ASIDE

~20% lower

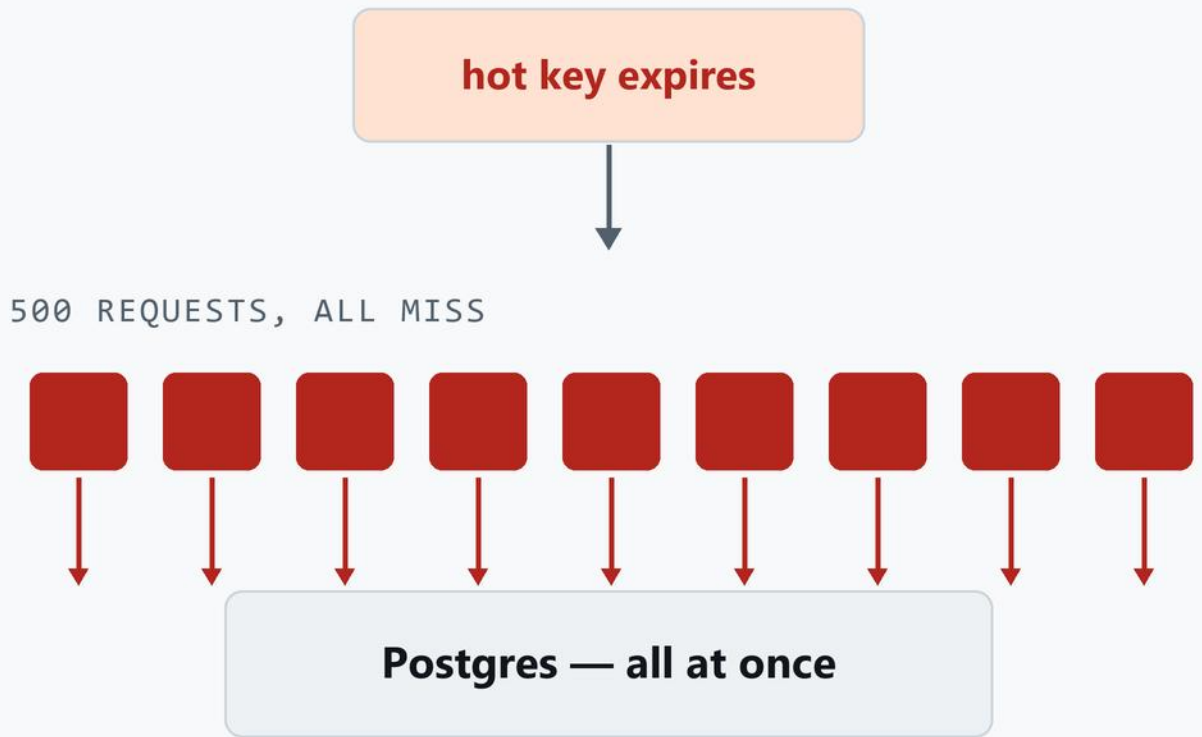
One hot read endpoint was hitting Postgres directly — for data that barely changed request to request.

BUT

**Cache-aside
doesn't protect
you from this.**

One expiring key. Same second.

CACHE STAMPEDE



No fix tested yet.

TTL jitter and request coalescing are the two I'd want to measure first.

The 20% is real.

**The stampede question
isn't answered yet.**

I'd rather post it that way than
pretend the second part is solved.

**If you've hit cache stampede in
production — jitter, locking,
request coalescing, or something
else? What actually worked?**